Django Middleware — Complete Beginner to Advanced Notes

1. What is Django Middleware?

Middleware is a layer in Django's request/response processing system that can process a request before it reaches the view and process the response before it reaches the client.

Simple flow:

```
Browser
   |
Request
   |
Middleware 1
   |
Middleware 2
   |
URL Resolver
   |
View
   |
Response
   |
Middleware 2
   |
Middleware 1
   |
Browser
```

Common use cases:

- Authentication infrastructure
- Logging
- Security
- Request tracking
- Rate limiting
- Custom headers
- Maintenance mode
- Multi-tenancy

- Request timing

2. Where is Middleware Configured?

Middleware is configured in the project's `settings.py` file inside the `MIDDLEWARE` list.

Example:

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

These are Django's built-in middleware classes.

3. Why is Middleware Used?

Middleware is generally used for functionality that needs to apply to many or nearly all requests.

Common examples:

- Request logging
- Security checks
- Session handling
- Request timing
- IP filtering
- Rate limiting
- Maintenance mode
- Custom response headers
- Tenant identification
- Request/response modification

4. Important Built-in Middleware

4.1 SecurityMiddleware

```
"django.middleware.security.SecurityMiddleware"
```

`SecurityMiddleware` handles various security-related HTTP behaviors based on Django's security settings.

For example, it can redirect HTTP requests to HTTPS when the appropriate setting is enabled.

4.2 SessionMiddleware

```
"django.contrib.sessions.middleware.SessionMiddleware"
```

This middleware provides session support.

Example:

```
request.session["name"] = "Faruk"

name = request.session.get("name")
```

Sessions allow data to persist across multiple requests from the same session.

4.3 CsrfViewMiddleware

```
"django.middleware.csrf.CsrfViewMiddleware"
```

This middleware provides Cross-Site Request Forgery (CSRF) protection.

Example:

```
<form method="post">
    {% csrf_token %}
    <button type="submit">Submit</button>
</form>
```

The CSRF token helps protect unsafe requests, such as POST requests, against CSRF attacks.

4.4 AuthenticationMiddleware

```
"django.contrib.auth.middleware.AuthenticationMiddleware"
```

This middleware makes the current user available through `request.user`.

Example:

```
if request.user.is_authenticated:
    print(request.user)
```

It requires `SessionMiddleware` to run before it.

4.5 MessageMiddleware

```
"django.contrib.messages.middleware.MessageMiddleware"
```

This middleware supports Django's messages framework.

Example:

```
from django.contrib import messages

messages.success(
    request,
    "Profile updated successfully."
)
```

4.6 XFrameOptionsMiddleware

```
"django.middleware.clickjacking.XFrameOptionsMiddleware"
```

This middleware helps protect against clickjacking by adding the configured `X-Frame-Options` response header.

5. Creating Custom Middleware

Suppose we want to print a message for every request.

Project structure:

```
project/
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
└── myapp/
    ├── middleware.py
    └── views.py
```

Create `myapp/middleware.py`:

```
class MyMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Request received")

        response = self.get_response(request)

        print("Response sent")

        return response
```

Register it in `settings.py`:

```
MIDDLEWARE = [
    # Existing Django middleware...
    "myapp.middleware.MyMiddleware",
]
```

6. Middleware `__init__()`

```
def __init__(self, get_response):
    self.get_response = get_response
```

`__init__()` is called when Django initializes the middleware instance.

It is generally used for:

- Middleware initialization
- Configuration
- Storing the next middleware handler

Important: Django normally initializes middleware once per application process, rather than once for every request.

Do not store request-specific mutable state on the middleware instance.

7. Middleware `__call__()`

```
def __call__(self, request):
    print("Before view")

    response = self.get_response(request)

    print("After view")

    return response
```

`__call__()` is the main method of modern Django middleware.

`self.get_response(request)` passes the request to the next middleware and eventually to the view.

After the downstream processing finishes, the response returns through the middleware layers.

8. Middleware Onion Model

Django middleware works like nested layers.

Request phase:

```
Request
   |
Middleware 1: Enter
   |
Middleware 2: Enter
   |
Middleware 3: Enter
   |
View
```

Response phase:

```
View
   |
Middleware 3: Exit
   |
Middleware 2: Exit
   |
Middleware 1: Exit
   |
Response
```

Outer middleware can execute code both before and after downstream processing.

9. Middleware Execution Order

The order of middleware in `MIDDLEWARE` is important.

Example:

```
MIDDLEWARE = [
    "myapp.middleware.MiddlewareA",
    "myapp.middleware.MiddlewareB",
]
```

Generally:

- Request processing enters middleware from top to bottom.
- Response processing returns through the layers in reverse order.

For the example:

```
Request:  A -> B -> View
Response: A <- B <- View
```

Exception handling, short-circuit responses, and response types can affect the exact execution path.

10. Middleware Short-Circuit

Middleware can stop a request from reaching the view by returning a response directly.

```
from django.http import HttpResponse


class MaintenanceMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        maintenance = True

        if maintenance:
            return HttpResponse(
                "Site under maintenance",
                status=503,
            )

        return self.get_response(request)
```

If maintenance mode is enabled, the view does not execute.

HTTP `503 Service Unavailable` is generally appropriate for temporary maintenance.

11. Middleware Hooks

There are two important concepts in Django middleware.

Modern middleware methods

```
__init__()
__call__()
```

Additional hooks

```
process_request()
process_view()
process_exception()
process_template_response()
process_response()
```

These additional hooks are commonly associated with `MiddlewareMixin` and Django's middleware hook system.

12. `process_view()`

`process_view()` runs after Django resolves the URL but before the view is executed.

Example:

```
from django.utils.deprecation import MiddlewareMixin


class ViewMiddleware(MiddlewareMixin):
    def process_view(
        self,
        request,
        view_func,
        view_args,
        view_kwargs,
    ):
        print("View:", view_func.__name__)
```

Parameters:

Parameter	Meaning
`request`	Current HTTP request
`view_func`	Resolved view function
`view_args`	Positional arguments for the view
`view_kwargs`	Keyword arguments for the view

Common uses:

- View-specific logging
- Request monitoring
- Special access checks

For normal authorization, Django's permission system and appropriate view-level checks are generally preferable.

13. When Should You Use `process_view()`?

Use it when you need information about the resolved view before execution.

Example use cases:

- Recording which view was called
- Applying special rules to selected views
- Monitoring view execution

It is not necessary to use `process_view()` for every authentication or permission requirement.

14. `process_exception()`

`process_exception()` can handle exceptions raised during downstream view processing.

Example:

```
import logging

from django.utils.deprecation import MiddlewareMixin

logger = logging.getLogger(__name__)


class ExceptionMiddleware(MiddlewareMixin):
    def process_exception(self, request, exception):
        logger.error(
            "View exception on %s",
            request.path,
            exc_info=(
                type(exception),
                exception,
                exception.__traceback__,
            ),
        )
```

Common uses:

- Error logging
- Monitoring
- Exception reporting

Returning a response from `process_exception()` can prevent the exception from propagating further. Returning `None` allows Django's normal exception handling to continue.

15. `process_template_response()`

This hook is useful when a view returns a `TemplateResponse`.

Example:

```
from django.utils.deprecation import MiddlewareMixin


class TemplateMiddleware(MiddlewareMixin):
    def process_template_response(self, request, response):
        response.context_data["site_name"] = "My Website"
        return response
```

The hook is not called for every type of response. It is specifically intended for responses supporting deferred template rendering.

16. `process_request()` and `process_response()`

These hooks are available through the legacy-style middleware interface supported by `MiddlewareMixin`.

Example:

```
from django.utils.deprecation import MiddlewareMixin


class MyMiddleware(MiddlewareMixin):
    def process_request(self, request):
        print("Request received")

    def process_response(self, request, response):
        print("Response sent")
        return response
```

`process_request()` can run before URL resolution.

`process_response()` processes the response on its way back through the middleware chain.

For new custom middleware, understanding the modern `__call__()` style is especially important.

17. Middleware Hooks Summary

Method	Purpose
`__init__()`	Initialize middleware
`__call__()`	Wrap request/response processing
`process_request()`	Process request before URL resolution
`process_view()`	Run after URL resolution, before the view
`process_exception()`	Handle eligible downstream exceptions

`process_template_response()`	Process a `TemplateResponse`
`process_response()`	Process the response

The behavior of additional hooks depends on middleware implementation and execution order.

18. Request Logging Middleware

Use Python's logging module instead of `print()` in production.

```
import logging

logger = logging.getLogger(__name__)


class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        logger.info(
            "Request: %s %s",
            request.method,
            request.path,
        )

        response = self.get_response(request)

        logger.info(
            "Response status: %s",
            response.status_code,
        )

        return response
```

This middleware records request methods, paths, and response status codes.

19. Request Time Measurement

Use `time.perf_counter()` to measure elapsed time.

```
import logging
import time

logger = logging.getLogger(__name__)


class TimerMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.perf_counter()

        response = self.get_response(request)

        duration = time.perf_counter() - start

        logger.info(
            "%s took %.4f seconds",
            request.path,
            duration,
        )

        return response
```

Use cases:

- Performance monitoring
- Slow endpoint detection
- Debugging

This measures synchronous downstream processing. Streaming response body generation may continue after the middleware returns.

20. Custom Response Header

Middleware can add a custom header to a response.

```
class CustomHeaderMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
```

```
    def __call__(self, request):
        response = self.get_response(request)

        response["X-App-Name"] = "My Django App"

        return response
```

Example response header:

```
X-App-Name: My Django App
```

21. IP Blocking Middleware

Simple example:

```
from django.http import HttpResponse


class BlockIPMiddleware:
    BLOCKED_IPS = {"192.168.1.10"}

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        ip = request.META.get("REMOTE_ADDR")

        if ip in self.BLOCKED_IPS:
            return HttpResponse(
                "Access Denied",
                status=403,
            )

        return self.get_response(request)
```

Important: In production, `REMOTE_ADDR` may contain the address of a reverse proxy instead of the original client.

Do not blindly trust `X-Forwarded-For` or similar headers. Configure trusted proxy infrastructure correctly before using forwarded client IP information.

22. Authentication / Login Check Middleware

A simple example:

```
from django.shortcuts import redirect
```

```
class LoginRequiredMiddleware:
    PUBLIC_PATHS = {
        "/login/",
        "/register/",
    }

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if (
            not request.user.is_authenticated
            and request.path not in self.PUBLIC_PATHS
        ):
            return redirect("login")

        return self.get_response(request)
```

Important: This example is intentionally simplified.

In real projects, Django's built-in tools are usually more appropriate:

```
from django.contrib.auth.decorators import login_required
```

For class-based views:

```
from django.contrib.auth.mixins import LoginRequiredMixin
```

Global login middleware must carefully handle admin, static/media assets, public pages, API endpoints, and login redirects.

If using `request.user`, ensure `AuthenticationMiddleware` is configured earlier in the middleware list.

23. API Authentication Middleware

Middleware can technically check an API key.

Conceptual example:

```
from django.http import JsonResponse


class APIKeyMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
```

```
    def __call__(self, request):
        if request.path.startswith("/api/"):
            api_key = request.headers.get("X-API-Key")

            if not api_key:
                return JsonResponse(
                    {"error": "Missing API key"},
                    status=401,
                )

            # Validate the key using a secure,
            # application-specific credential system.

        return self.get_response(request)
```

Important: This is not complete authentication until the key is actually validated. Never hard-code production API keys. Use environment variables or a secret-management system, and consider database-backed credentials where appropriate. For Django REST Framework APIs, DRF authentication classes are often a better place for endpoint authentication.

24. JWT Authentication Middleware

JWT authentication can be implemented manually, but established authentication packages are generally preferable.

Conceptual flow:

```
Request
   |
Authorization Header
   |
Extract Bearer Token
   |
Verify Signature and Claims
   |
Check Expiration, Issuer and Audience
   |
Identify User
   |
Authentication / Authorization
   |
View
```

Simply calling `jwt.decode()` does not create a complete authentication system.

You must also consider:

- Signature verification
- Token expiration
- Issuer and audience
- Token type
- Error handling
- User lookup
- Authorization
- Revocation strategy, if required

In Django REST Framework, use a suitable authentication class or established JWT package.

25. Rate Limiting Middleware

Rate limiting restricts the frequency of requests from a client, user, or IP.

Concept:

```
Client Request
      |
Rate Limiter
      |
Within Limit? ---- No ----> HTTP 429
      |
     Yes
      |
     View
```

Example response:

```
from django.http import JsonResponse

response = JsonResponse(
    {"error": "Too many requests"},
    status=429,
)
```

A dictionary such as:

```
requests = {}
```

is not a production-ready rate limiter.

Potential problems:

- Multiple worker processes
- Multiple servers
- Process restarts
- Memory usage
- Race conditions
- Expiration and cleanup
- Distributed state

For production, consider Redis-backed rate limiting, Django REST Framework throttling, or a suitable established library.

26. Maintenance Mode Middleware

```
from django.http import HttpResponse


class MaintenanceMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        maintenance = True

        if maintenance:
            return HttpResponse(
                "Site under maintenance",
                status=503,
            )

        return self.get_response(request)
```

In a real application, the maintenance flag should come from an appropriate configuration or feature-flag mechanism.

Ensure administrators and health-check endpoints can be handled appropriately.

27. Multi-Tenant Middleware

In SaaS applications, tenants can be identified by subdomains.

Example:

```
company1.example.com
company2.example.com
```

Basic conceptual middleware:

```
class TenantMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        host = request.get_host()
        request.tenant_host = host

        return self.get_response(request)
```

This only attaches a host value to the request; it does not implement tenant isolation.

A production multi-tenant system needs:

- Validated tenant lookup
- Subdomain validation
- Database isolation strategy
- Tenant-specific data filtering
- Authorization
- Cache isolation
- Protection against cross-tenant data access

28. Middleware vs Decorator vs Signal

Feature	**Middleware**	**Decorator**	**Signal**
Scope	Request/response lifecycle	Specific function or view	Application event
Trigger	Request processing	Function/view call	Specific event
Common use	Logging, headers, monitoring	Login protection, view-	Model/application events

		specific behavior	
Example	Request logging	`login_required`	`post_save`

Easy rule:

- Every request needs logging: use middleware.
- Only one view needs login protection: use a decorator.
- Something should happen when a model is saved: consider a signal.

29. Middleware Example

```
class LogMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print(request.path)

        response = self.get_response(request)

        return response
```

Register it in `settings.py`:

```
MIDDLEWARE = [
    # Existing middleware...
    "myapp.middleware.LogMiddleware",
]
```

30. Decorator Example

```
from functools import wraps


def my_decorator(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        print("Before view")

        response = view_func(
```

```
            request,
            *args,
            **kwargs,
        )

        print("After view")

        return response

    return wrapper
```

Usage:

```
@my_decorator
def home(request):
    # Return an appropriate HTTP response
    ...
```

A decorator is useful when the behavior applies to a particular function or view.

31. Signal Example

Signals can be used for model or application events.

```
from django.db.models.signals import post_save
from django.dispatch import receiver

from .models import Product


@receiver(post_save, sender=Product)
def product_saved(
    sender,
    instance,
    created,
    **kwargs,
):
    if created:
        print("New product:", instance.name)
```

The handler runs when the relevant `Product` save signal is sent.

Use signals carefully. For critical business workflows, explicit service-layer logic may be easier to maintain and test.

32. Async Middleware

Modern Django supports asynchronous middleware.

Example:

```
import logging

logger = logging.getLogger(__name__)


class AsyncMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    async def __call__(self, request):
        logger.info("Before downstream processing")

        response = await self.get_response(request)

        logger.info("After downstream processing")

        return response
```

This example is intended for an asynchronous middleware execution path.

Important: Using async middleware does not automatically make an application faster.

Performance depends on:

- I/O workload
- Async-compatible views
- Database behavior
- External API calls
- Middleware design
- Server configuration

Middleware must be compatible with the sync/async execution mode in which Django runs it.

33. Middleware and WebSockets

When using Django Channels, WebSocket requests are handled through the ASGI/Channels stack.

Traditional Django HTTP middleware and Channels WebSocket middleware are not exactly the same.

For WebSocket authentication, Channels provides tools such as:

```
AuthMiddlewareStack
```

Conceptual architecture:

```
Browser
   |
WebSocket
   |
ASGI
   |
Channels Middleware
   |
Consumer
```

34. Production-Level Logging Middleware

```
import logging
import time

logger = logging.getLogger(__name__)


class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.perf_counter()

        response = self.get_response(request)

        duration = time.perf_counter() - start

        logger.info(
            "%s %s %s %.3fs",
            request.method,
            request.path,
            response.status_code,
            duration,
        )

        return response
```

Example log:

```
GET /products/ 200 0.125s
POST /checkout/ 201 0.240s
```

Configure Python logging appropriately for your deployment environment.

Note: If downstream processing raises an exception, this simple middleware will not log a completed response. Add deliberate exception handling if you need exception timing or error logging.

35. Request Tracking with Middleware

```
import logging

from django.utils import timezone

logger = logging.getLogger(__name__)


class RequestTrackingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        logger.info(
            "Request method=%s path=%s time=%s",
            request.method,
            request.path,
            timezone.now(),
        )

        response = self.get_response(request)

        return response
```

Be careful not to log sensitive information.

Never log passwords, access tokens, refresh tokens, OTP codes, session secrets, or payment-card information.

36. Custom Security Headers

Middleware can add security-related headers.

```
class SecurityHeadersMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
```

```
    def __call__(self, request):
        response = self.get_response(request)

        response["X-Frame-Options"] = "DENY"
        response["X-Content-Type-Options"] = "nosniff"

        return response
```

Before adding headers manually, review Django's built-in security middleware and settings.

Modern browser security should also consider Content Security Policy (CSP), output escaping, secure cookie settings, HTTPS, and other appropriate security controls.

The old `X-XSS-Protection` header should not be treated as a primary modern browser security mechanism.

37. Complete Middleware Flow

A simplified flow:

```
Browser Request
      |
SecurityMiddleware
      |
SessionMiddleware
      |
CommonMiddleware
      |
CsrfViewMiddleware
      |
AuthenticationMiddleware
      |
Custom Middleware
      |
URL Resolution
      |
View
      |
Response
      |
Custom Middleware
      |
```

```
AuthenticationMiddleware
      |
Other Middleware
      |
SecurityMiddleware
      |
Browser
```

The actual execution path depends on middleware order, exceptions, response types, and Django configuration.

38. Middleware Short-Circuit Flow

```
Request
   |
Middleware
   |
Check Condition
   |
   +---- Block ----> Return Response
   |
   +---- Allow ----> Next Middleware
                          |
                        View
```

Possible uses:

- Maintenance mode
- IP blocking
- Certain global access checks
- Request rejection based on validated conditions

For API authentication and permissions, framework-specific authentication and authorization tools are usually more suitable.

39. Important Middleware Concepts

39.1 Order Matters

```
MIDDLEWARE = [
    "myapp.middleware.MiddlewareA",
    "myapp.middleware.MiddlewareB",
]
```

`MiddlewareA` is the outer layer and `MiddlewareB` is the inner layer.

39.2 Not Everything Belongs in Middleware

Avoid placing complex business logic in middleware.

Examples better suited to services or domain logic:

- Order calculations
- Payment processing
- Complex business rules
- Transaction workflows

Middleware should primarily handle cross-cutting concerns.

40. Good Use Cases for Middleware

- Logging
- Security
- Request tracing
- Request timing
- Custom headers
- Tenant identification
- Rate limiting
- Maintenance mode
- Global request metadata

41. Good Middleware Design Principles

Middleware should be:

- Small
- Focused
- Reusable
- Predictable
- Fast
- Easy to test

Avoid putting hundreds of lines of unrelated business logic in a single middleware class.

42. `settings.py` Example

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
```

```
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
    "myapp.middleware.LoggingMiddleware",
]
```

The correct position of custom middleware depends on its dependencies and intended behavior.

For example, middleware that needs `request.user` should normally run after `AuthenticationMiddleware`.

43. Middleware Project Structure

```
project/
├── manage.py
├── project/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
└── myapp/
    ├── __init__.py
    ├── middleware.py
    ├── models.py
    └── views.py
```

`myapp/middleware.py`:

```
class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print(request.path)

        return self.get_response(request)
```

`settings.py`:

```
MIDDLEWARE = [
    # Existing Django middleware...
    "myapp.middleware.LoggingMiddleware",
]
```

---

44. Interview Questions and Answers

Q1. What is Django Middleware?

Middleware is a layer in Django's request/response processing system that processes requests and responses around view execution.

Q2. Where is Middleware configured?

In `settings.py`, inside the `MIDDLEWARE` list.

Q3. What are the main methods of modern Middleware?

`__init__()` and `__call__()`.

Q4. When is `__init__()` executed?

When Django initializes the middleware instance.

Q5. When is `__call__()` executed?

During request processing, normally once for each request handled by that middleware.

Q6. What is `process_view()`?

A middleware hook that runs after URL resolution but before the view executes.

Q7. What is `process_exception()`?

A hook that can handle or log eligible exceptions raised during downstream view processing.

Q8. What is `process_template_response()`?

A hook used to process a `TemplateResponse` before its template is rendered.

Q9. Can middleware block a request?

Yes. Middleware can return a response directly without passing the request further down the chain.

Q10. What is the difference between Middleware and Decorator?

Middleware operates around request/response processing, while decorators usually wrap specific functions or views.

Q11. What is the difference between Middleware and Signal?

Middleware operates in the request/response lifecycle. Signals trigger handlers in response to specific application events.

Q12. What is Middleware execution order?

Request processing generally goes from top to bottom. Response processing returns through the middleware layers in reverse order.

Q13. Why does middleware order matter?

Middleware may depend on attributes or processing performed by earlier middleware. For example, `AuthenticationMiddleware` depends on `SessionMiddleware`.

Q14. Can middleware access `request.user`?

Yes, if `AuthenticationMiddleware` has already processed the request.

Q15. Does async middleware automatically improve performance?

No. Benefits depend on workload, asynchronous I/O, execution mode, and the rest of the application.

Q16. Is middleware appropriate for every business rule?

No. Complex business logic is usually better placed in services, views, or domain-specific components.

Q17. What is `MiddlewareMixin`?

It provides compatibility with Django's legacy-style middleware hooks such as `process_request()` and `process_response()`.

Q18. How do you register custom middleware?

Add its Python import path to the `MIDDLEWARE` list in `settings.py`.

Q19. How should request rate limiting be implemented in production?

Use a suitable shared backend or established rate-limiting implementation that supports concurrent workers and servers.

Q20. How can middleware improve monitoring?

It can record request metadata, status codes, processing time, and correlation IDs while avoiding sensitive information.

---

45. Quick Revision

Middleware

```
Request -> Middleware -> View -> Middleware -> Response
```

Main Methods

```
__init__()
__call__()
```

Additional Hooks

```
process_request()
process_view()
process_exception()
process_template_response()
process_response()
```

Middleware vs Decorator vs Signal

```
Middleware = Request/Response Level
Decorator  = Specific Function/View Level
Signal     = Event Level
```

Real-World Use Cases

Use Case	Middleware Role
Logging	Record request and response metadata
Security	Apply security-related behavior
Authentication infrastructure	Make authentication information available
Rate limiting	Restrict request frequency

Monitoring	Measure request processing time
Maintenance mode	Temporarily reject requests
Multi-tenancy	Identify tenant context
Custom headers	Add response headers
Request tracking	Track request metadata

Final Mental Model

```
                BROWSER
                   |
                REQUEST
                   |
            MIDDLEWARE 1
                   |
            MIDDLEWARE 2
                   |
            MIDDLEWARE 3
                   |
                VIEW
                   |
               RESPONSE
                   |
            MIDDLEWARE 3
```

```
                |
          MIDDLEWARE 2
                |
          MIDDLEWARE 1
                |
             BROWSER
```

One-Line Definitions

- **Middleware:** A processing layer in Django's request/response lifecycle.
- **`__init__()`:** Initializes a middleware instance.
- **`__call__()`:** Wraps downstream request/response processing.
- **`process_view()`:** A hook that runs before the view executes.
- **`process_exception()`:** Handles eligible downstream exceptions.
- **`process_template_response()`:** Processes `TemplateResponse` objects.
- **MiddlewareMixin:** Supports legacy-style middleware hooks.
- **Decorator:** Wraps or modifies the behavior of a specific function or view.
- **Signal:** Executes a handler when a specific event occurs.
- **ASGI:** An interface for asynchronous-capable Python web applications and protocols.
- **Channels:** A Django ecosystem extension for WebSockets and other asynchronous communication.
- **Redis:** An in-memory data store commonly used for caching, shared state, and channel layers.

Final Rule to Remember

```
Global Request/Response Concern -> Middleware
Specific View Concern           -> Decorator
Specific Application Event      -> Signal
```

Practice custom middleware, execution order, short-circuiting, request logging, and async middleware to build a strong understanding from beginner to advanced level.